

max

算法标签：分析性质；前缀和；*st* 表

先考虑如何求取答案，不难发现答案是两边之和与任意某一位对应位置相加取最大的一个。（这是一个显然的必要条件，具体严格证明可以建立最大流模型证明结论，但一个更感性的分析类似于绝对众数的分析）

因此 n, q 比较小的时候可以直接暴力枚举做到 $O(n \times q)$ 。

分析一些性质，对于 $a_i + b_j$ 形式的贡献，要么 a_i 是区间 $[l_1, r_1]$ 中唯一的最大值，要么 b_j 是区间 $[l_2, r_2]$ 中唯一的最大值，要么 $a_i \geq \max(b_{l_2}, \dots, b_{r_2})$ 且 $b_j \geq \max(a_{l_1}, \dots, a_{r_1})$ 。

而对于第三种情况，如果两边最大值不等则不可能产生贡献，否则此时要么可能的位置已经被第一二种情况囊括，要么当前某一侧的最大值有多个，在最大值有多个的情况下 $a_i + b_j$ 不可能作为最大值，因此我们只需要考虑两边各自的和以及第一二种情况即可。使用前缀和以及 *st* 表维护这个东西，时间复杂度 $O(n \log n + q)$ 。

sum

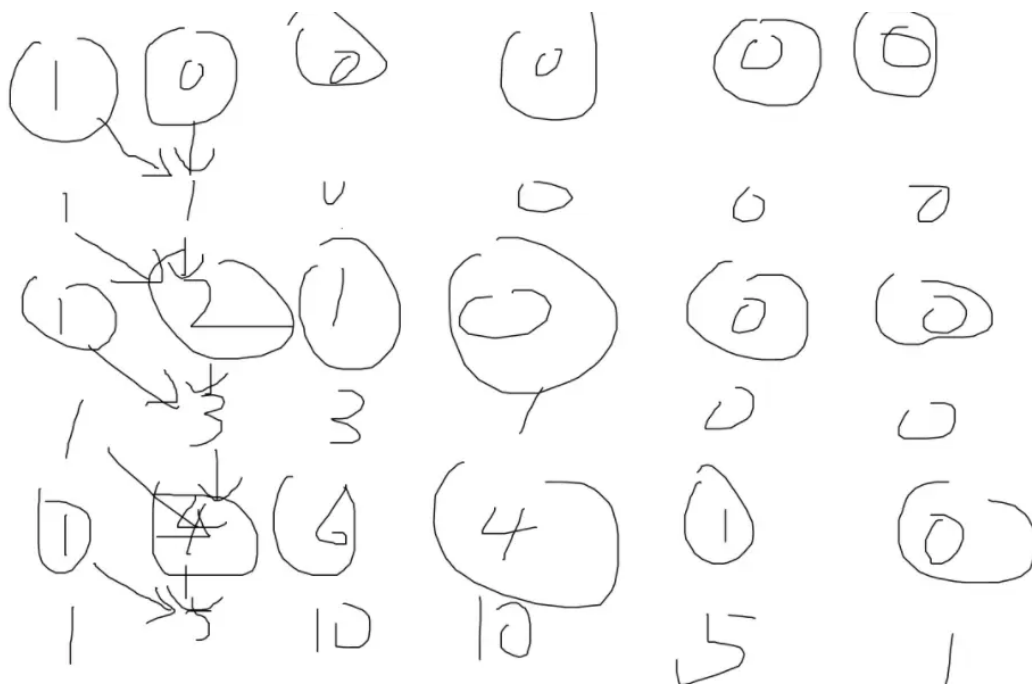
算法标签：组合数递推；根号分治；主元法

感觉是比较套路的题。

首先让 n 变为满足 $k|x+1$ 的最大的小于 n 的数 x 。然后对于边界上的点单独处理，也就是每次询问直接加上被减掉的。

定义 $g_{x,r}$ 表示 $\sum_{i=0}^n [i \bmod k = r] f_{i,x}$ 。

为了形象的表达，我们画出组合数的图。



可以看出对于 $j \in [1, k-1]$, $g_{i,j} = g_{i-1,j-1} + g_{i-1,j}$ 。但是此时相当于是一个 k 个变量, $k-1$ 个方程求解, 不能求出解。不过注意到当 $k=0$ 时在特殊处理边界情况后也可以写出类似的方程。此时就变成了 k 个变量, k 个方程求解。直接高斯消元是 k^3 的, 使用主元法可以做到 $O(k)$ 求解。

不过更简单的, 不用特殊处理的写法是: 注意到恒等式 $\sum_{i=0}^n \binom{i}{x} = \sum_{i=0}^{k-1} g_{x,i} = \binom{n+1}{x+1}$, 这样同样可以变成对 k 个变量, k 个方程求解。

这样可以做到 $O(nk)$ 。不过显然直接暴力显然是 $O(\frac{nq}{k})$ 的, 于是平衡一下可以做到 $O(n\sqrt{q})$ 。

distance

算法标签：线段树分治；迪杰斯特拉算法

本题非原创，原题链接：[最短路径 - Problem - QQJ.ac](#)

先考虑一些部分分。

注意到 n, q 都很小可以暴力枚举 p_i 跑弗洛伊德算法，这也可以通过 $p_i = 1$ 的部分分。

而 q 较小的情况可以考虑直接跑 $O(n^2)$ 的迪杰斯特拉算法，可以通过 $1 \leq q \leq 10^3$ 。

考虑 n, q 更大的情况。注意到这种原图“扣去”一个点是线段树分治来刻画的。

具体的，我们令线段树上 p 号节点所代表的区间为 $[x, y]$ ；那么，我们可以设计状态 $f_{p,i,j}$ 表示在只能经过 $[1, x) \cup (y, n]$ 中的点时， i 号点到达 j 号点的最短路长度。状态数量是 $O(n^3)$ 级别的。

再考虑如何计算 f 。注意到 f 是可以从线段树上父亲节点部分继承的。具体的，令 ls 为 p 的左儿子， rs 为 p 的右儿子， mid 为 $[l, r]$

的中点，那么我们先令 $f_{ls} = f_p, f_{rs} = f_p$ ，然后将 $[l, mid]$ 中的点加入到 f_{rs} 中、 $[mid + 1, r]$ 中的点加入到 f_{ls} ，这样加入的总点数即为 $O(n \log n)$ 个。

最后，我们考虑每加入一个新的点 x 如何维护 f_p 的变化。可以先计算其他点到 x 以及 x 到其他点的距离，这可以通过迪杰斯特拉算法简单计算；对于点集中其他两点 a, b 的距离，则可以通过 $f_{p,a,x} + f_{p,x,b}$ 更新 $f_{p,a,b}$ 。至此，我们已经用 $O(n^2)$ 的复杂度完成单个点的加入。

因此，算法总时间复杂度为 $O(n^3 \log n + q)$ ，空间复杂度为 $O(n^3)$ 。

Bonus: 思考如何降低算法的空间复杂度。

bst

算法标签：二叉搜索树；区间 dp ；决策单调性优化 dp 。

弱化版链接：[K](#)

首先考虑观察插入操作的性质。注意到，对于每个插入的 x ，找到 bst 中权值小于 x 的权值最大值 $last$ 和权值大于 x 的权值最小值 $next$ ，可以发现 x 必然插入在这两个子树之一，而作为谁的儿子只与谁后被插入有关。

因此，可以按照 $a_1, a_2, \dots, a_r$ 把 $a_{r+1}, a_{r+2}, \dots, a_n$ 分成一些段，其中每一段的子树形态是固定且独立的，这个可以通过 set 预处理。

而 $a_1, a_2, \dots, a_{l-1}$ 形态也固定，因此真正变化的是 $a_l, a_{l+1}, \dots, a_r$ 的结构。观察到此处的前驱后继关系，不难设计基于值域的区间 dp ，简单转移可以做到 $O((r-l+1)^3)$ ，其中状态 $O((r-l+1)^2)$ 。

注意到此 dp 数组满足四边形不等式，因此可以每一层整体二分做到 $O((r-l+1)^2 \log(r-l+1))$ ；进一步的，可以利用决策单调性优化 dp 做到 $O((r-l+1)^2)$ 。